21. The Method of State Analysis

Solving SC Computation Problems

The way that circuits with memory work can be unintuitive, and solving SC computation problems is sometimes difficult. What follows is a step-by-step method for turning SC problems into discrete sequences of CC problems. To do this, we'll have to conceptualize SCs in terms of abstract *states*— a concept will examine in greater depth in the coming chapters. Analyzing SC's in terms of states gives you a distinctive understanding of their core mechanism.

Histories and States

Over the course of the history of a machine, it will enter into various fixed states, that often repeat. Consider various sequences for the SC tripwire, and its two characteristic states.

…	t5	t4	t3	t2	t1	
…	0	0	1	0	0	IN
…	1	1	1	0	0	OUT

tripped state un-tripped state

…	t5	t4	t3	t2	t1	
…	0	0	1	1	0	IN
…	1	1	1	1	0	OUT

tripped state un-tripped state

These enduring states can be further analyzed into instantaneous states of the machine that repeat in time.

…	t5	t4	t3	t2	t1	
…	0	0	1	1	0	IN
…	1	1	1	1	0	OUT

tripped state tripped state tripped state tripped state un-tripped state

The idea behind state analysis is to describe a sequential circuit in terms of the characteristic inputs and outputs of a state over time, and the conditions under which one state leads to another, or simply repeats.

Example 1: Trip-Wire

Design a **trip-wire** which sets off an "alarm" (outputs constant 1's) as soon as it reads a 1 for the first time.

Step 1: Sequential input-output table. Start by making a left-to-right table of local inputs and outputs over time. (A top-to-bottom table will also work, but the horizontal layout will be helpful when we get to numerical computation.). You may want to make a few of these, for variation.

One sequence…

…	t5	t4	t3	t2	t1	
…	0	0	0	0	0	IN
…	0	0	0	0	0	OUT

Another sequence…

…	t5	t4	t3	t2	t1	
…	0	0	1	0	0	IN
…	1	1	1	0	0	OUT

Yet another…

…	t5	t4	t3	t2	t1	
…	0	0	1	1	0	IN
…	1	1	1	1	0	OUT

Step 2: State description. Consider how many "states" the circuit has, or overall ways of responding to input and output. (We'll define this idea more precisely later.) In this case, the machine has two states— before the first 1, and after the first 1. Write these down in words.

State 0

If IN=0, OUT= 0. Repeat State 0.

If IN=1, OUT=1. Go to State 1.

State 1

If IN=0, OUT= 1. Repeat State 1.

If IN=1, OUT=1. Repeat State 1.

As a rule, start with State 0, and count up from there. In general, each state description will have the following schematic form:

State X

If IN=A, then OUT= B. Go to state Y.

If IN=C, then OUT=D. Go to State Z.

Step 3: State tables. Next, make construct an input-output table for each state. But when you do this, include extra input and output columns for memory, M_{IN} and M_{OUT}. M_{IN} is the bit coming *in* from memory, and M_{OUT} is to the bit going *out* into memory.

A single memory register has two states (0 and 1). So for the current problem, we can assume that State 0 is defined by a 0 in memory, and State 1 is defined by a 1 in memory. Since M_{IN} is the digit coming in from memory at the beginning of a new cycle, State 0 corresponds to $M_{IN} = 0$, and State 1 corresponds to $M_{IN} = 1$. In the same way, an instruction to move State 0 corresponds to $M_{OUT} = 0$, and an instruction to move State 1 corresponds to $M_{OUT} = 1$.

State 0

If IN=0, OUT= 0. Repeat State 0.

If IN=1, OUT=1. Go to State 1.

State 1

If IN=0, OUT= 1. Repeat State 1.

If IN=1, OUT=1. Repeat State 1.

State 0

M_{IN}	IN	OUT	M_{OUT}
0	0	0	0
0	1	1	1

State 1

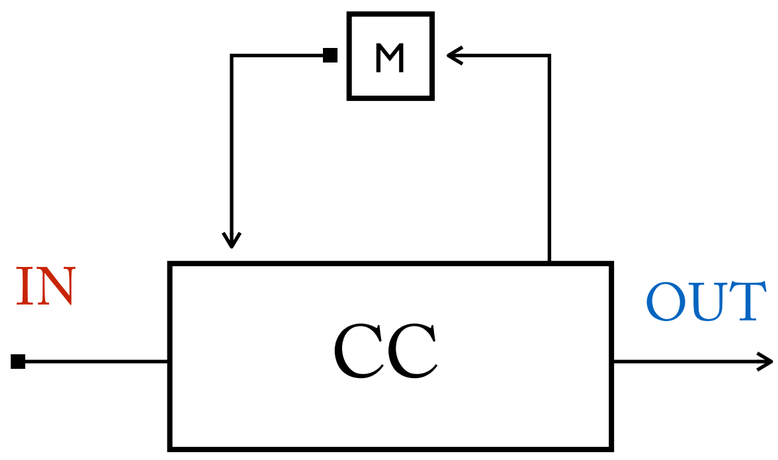
M_{IN}	IN	OUT	M_{OUT}
1	0	1	1
1	1	1	1

Step 4: CC design. Assemble these states into a single input output table, and design a CC which computes this table. And design a CC to solve it.

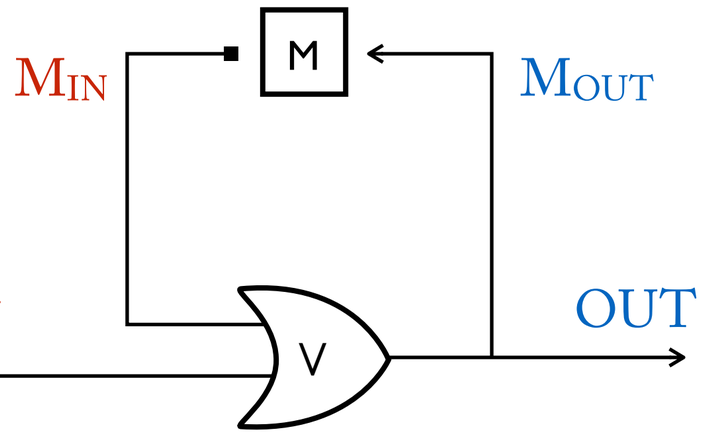
M_{IN}	IN	OUT	M_{OUT}
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	1



Step 5: SC design. Convert your CC design into a SC, follwing the schematic layout of an SC:



SC schematic layout



SC tripwire

+1 T: Step by Step

Design an SC which computes +1 T.

Step 1. Start by making a global-input output table, the way you might for any numerical function.

IN	OUT
0	1
1	11
11	111
111	1111
…	…

Remember you can always add 0's to the left, without changing the global inputs and outputs.

IN	OUT
0000	0001
0001	0011
0011	0111
0111	1111
…	…

To get a sense for what's going on here, make a left-to-right table of local inputs and outputs over time. You will want to look at a few different variations. Key cases to always check for are x=0 and x≠0.

+1 T where x=0

t5	t4	t3	t2	t1	
…	0	0	0	0	IN
…	0	0	0	1	OUT

+1 T where x=2

t5	t4	t3	t2	t1	
…	0	0	1	1	IN
…	0	1	1	1	OUT

Step 3 & 4. Divide the machine up into states. This step isn't formulaic; you'll have to think about how the computation intuitively works. Then make an I/O table for each state.

State 0

If IN=0, OUT= 1. Go to State 1.
If IN=1, OUT=1. Repeat State 0.

State 1

If IN=0, OUT= 0. Repeat State 1.
If IN=1… **never happens!**

State 0

I	M→	O	→M
0	0	1	1
1	0	1	0

State 1

I	M→	O	→M
0	1	0	1
1	1		

Now combine these two state descriptions so it looks like a familiar input-output table:

I	M→	O	→M
0	0	1	1
1	0	1	0
0	1	0	1
1	1		

Step 5. Design a circuit which handles the table from Step 4.

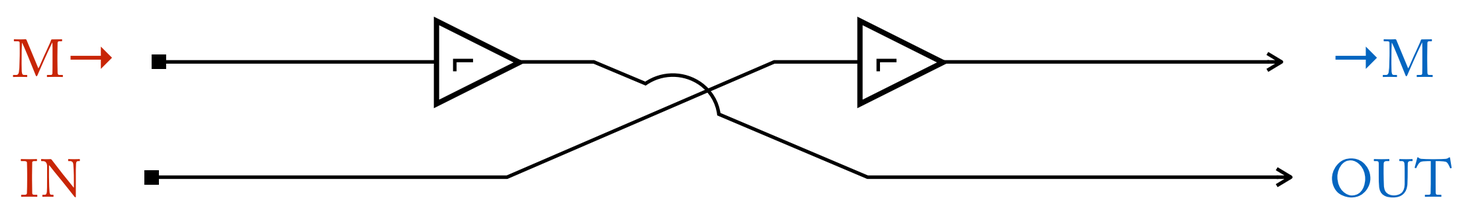
I	M→	O	→M
0	0	1	1
1	0	1	0
0	1	0	1
1	1		

Typically it's useful to solve for the O and →M columns separately. Here I'd note two patterns:

(1) Inverting the M→ value always gives you the O value.

(2) Inverting the I value always gives you the →M value.

We can draw this up as a simple circuit:



We now interpret "M→" as *reading from memory*, and we interpret "→M" as *writing to memory*. The result is our final circuit.

